

CO4 – Assessment Tool 1: Industry Problem Solving Task

Question 1: Smartphone Performance Optimization

Problem: A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

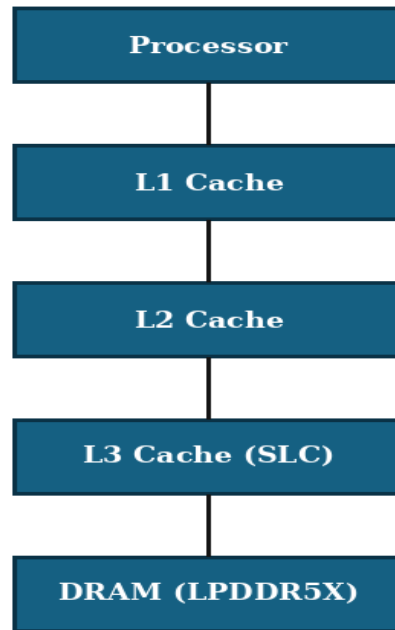
1. Understanding the Problem

App-switch lag on a smartphone is usually a memory-hierarchy symptom rather than a CPU-clock-speed problem. When the user switches apps, the working set (instructions, UI state, heap objects) of the new app is not present in the fast on-core caches, so the pipeline stalls on cache misses that must be resolved by DRAM, which is orders of magnitude slower than SRAM-based caches. If DRAM is also under pressure (multiple background apps held in memory), the OS may have to swap/compress pages, adding further delay.

2. Role of Each Level

Level	Typical Size	Latency	Role in App Switching
L1 (I/D)	32–64 KB per core	~1–4 cycles	Holds hot instructions/data of the foreground thread; too small to hold a new app's working set, so it misses immediately on switch.
L2	256 KB–1 MB per core/cluster	~10–20 cycles	Buffers recently evicted L1 lines; helps warm-restart an app that was recently backgrounded (still resident).
L3 (SLC)	2–8 MB shared	~30–50 cycles	Shared across big.LITTLE clusters; keeps commonly used OS/framework code resident, reducing DRAM trips for shared libraries.
DRAM (LPDDR)	4–16 GB	~100–300 cycles	Holds full app image and heap; a cold app requires paging its code/data in from DRAM (or flash if evicted), which dominates switch latency.

3. Proposed Hierarchical Redesign



- Increase L2 private cache per core cluster and adopt a victim-cache policy so recently backgrounded apps stay resident longer instead of being evicted early.
- Use a larger, shared System-Level Cache (L3/SLC) with app-aware partitioning so the foreground app is not starved of cache lines by background services (cache partitioning / QoS-aware replacement).
- Adopt LPDDR5X/LPDDR5 DRAM with higher bandwidth and lower access latency, paired with an on-chip memory controller that prioritizes foreground-app requests.
- Use hardware/OS cooperative prefetching: predict the next likely app (based on usage pattern) and prefetch its critical working set from flash into DRAM and DRAM into L3 before the user taps the icon.
- Employ compressed RAM (zRAM) for backgrounded apps so more apps stay in DRAM instead of being killed and cold-started, avoiding the most expensive miss path (flash read).

4. Expected Outcome

Together, a deeper/partitioned cache hierarchy plus faster DRAM and predictive prefetching reduces the number of full DRAM/flash round-trips needed on an app switch, cutting perceived lag and improving sustained multitasking throughput.

Question 2: Cloud Server Memory Bottleneck

Problem: A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

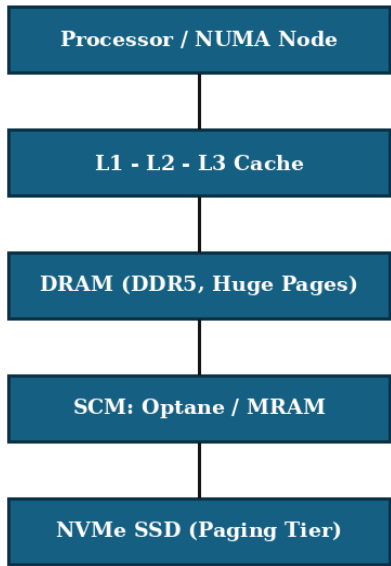
1. Understanding the Problem

Database workloads are memory-intensive and have large, often random-access working sets (indexes, buffer pools) that exceed on-chip cache capacity. Latency can originate from two different mechanisms: (a) cache misses that force accesses out to DRAM, and (b) virtual-memory page faults that force accesses out to secondary storage (disk/SSD) when the working set exceeds physical RAM.

2. Cache Hierarchy vs Virtual Memory Paging

Aspect	Cache Hierarchy (L1–L3)	Virtual Memory Paging
Granularity	Cache lines (64 B)	Pages (4 KB–2 MB)
Managed by	Hardware	OS + MMU/TLB (hardware-assisted)
Miss penalty	~4–300 cycles (to DRAM)	Thousands to millions of cycles (disk/SSD I/O)
Typical cause of DB latency	Random index lookups exceeding L2/L3 capacity	Buffer pool larger than physical RAM, causing swapping
Mitigation	Larger/associative caches, prefetching	More RAM, better page replacement, huge pages

3. Recommendations



- Right-size the L3/last-level cache and use NUMA-aware placement so each database node accesses local DRAM, avoiding costly remote-node latency.
- Use huge pages (2 MB/1 GB) for the database buffer pool to reduce TLB misses and page-table walk overhead, a common hidden cause of query latency.
- Increase physical DRAM (or use DDR5 for higher bandwidth) so the working set fits in memory and paging to disk is minimized; where budgets are constrained, use NVMe SSDs as a much faster fallback than spinning disk for the paging tier.
- Introduce a tiered memory architecture: DRAM as the hot tier, and Storage-Class Memory such as Intel Optane / MRAM-backed persistent memory as a warm tier, reducing the latency gap when a full page fault to SSD is unavoidable.
- Use application-level caching (e.g., in-memory key-value caches) tuned to sit logically 'above' the OS page cache, reducing pressure on the virtual memory system.

4. Expected Outcome

Reducing cache misses shrinks the microsecond-scale latency component, while minimizing page faults removes the millisecond-scale tail latency that dominates worst-case query response times, together yielding more consistent, lower-latency database performance.

Question 3: Autonomous Vehicle Data Processing

Problem: An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

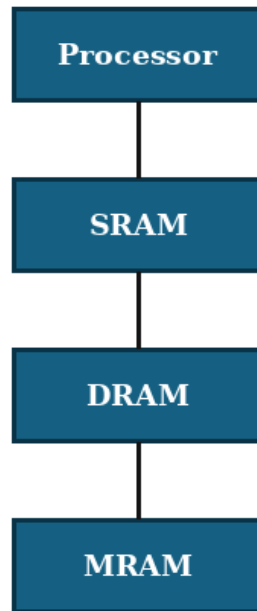
1. Understanding the Problem

Autonomous driving pipelines (LiDAR point clouds, camera frames, radar, sensor fusion, path planning) require deterministic, low-latency, high-bandwidth memory access with strict real-time guarantees — a missed deadline can be safety-critical. The design must balance raw speed (SRAM), capacity/bandwidth (DRAM), and non-volatility/reliability (MRAM) for logging and safe-state recovery.

2. Technology Evaluation

Technology	Latency	Bandwidth	Volatility	Best Fit in AV Pipeline
SRAM	Sub-ns to ~1 ns	Very high, on-chip	Volatile	L1/L2 cache and scratchpad for real-time sensor-fusion kernels needing deterministic access
DRAM (e.g., GDDR6/HBM)	~50–100 ns	High (100s of GB/s with HBM)	Volatile	Frame buffers, point-cloud staging, neural-network weights/activations during inference
MRAM	~10–30 ns (comparable to SRAM/fast DRAM)	Moderate	Non-volatile	Safety-critical state logging, black-box event data, instant-on boot data that must survive power loss

3. Proposed Memory Hierarchy



- On-chip SRAM scratchpads and L1/L2 caches tightly coupled to each processing core (vision DSP, fusion core) for deterministic, single-cycle-class access to actively processed sensor windows.
- High-bandwidth DRAM (HBM2E/GDDR6) as the shared memory for CNN/inference workloads and multi-sensor frame buffers, since bandwidth — not just latency — is the bottleneck for parallel image/point-cloud processing.
- Non-volatile MRAM tier for critical state (last-known-good trajectory, fault logs, calibration data) so that on power interruption or a system reset, the vehicle can recover safely without waiting on flash boot latency.
- Use a real-time memory controller with priority arbitration so safety-critical fusion/planning threads always get guaranteed bandwidth ahead of infotainment or logging traffic.

4. Expected Outcome

This tiered SRAM–DRAM–MRAM hierarchy delivers the low, predictable latency SRAM provides for real-time control loops, the bandwidth DRAM provides for sensor/AI throughput, and the resilience MRAM provides for safety-critical persistence — together meeting the hard real-time and safety requirements of autonomous driving.

Question 4: AI Inference Edge Device

Problem: An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

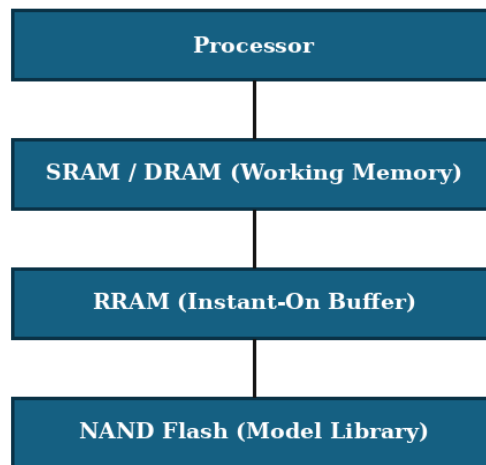
1. Understanding the Problem

Edge AI devices (smart cameras, IoT inference nodes) are constrained by power and cost, but still need to load large model weights quickly at boot or on model switch, then run inference with low-latency memory access. The key trade-off is between NAND Flash's high density/low cost/non-volatility, DRAM's speed, and RRAM's emerging balance of non-volatility with faster access than flash.

2. Technology Comparison

Technology	Latency	Power	Non-volatile	Endurance	Role
NAND Flash	~10s of μ s (page read)	Low (idle), higher for write/erase	Yes	Limited P/E cycles	Bulk, low-cost storage of model weights and firmware
DRAM	~50–100 ns	Needs constant refresh power	No	Practically unlimited	Working memory for active model weights/activations during inference
RRAM	~10–50 ns	Very low (no refresh, low write energy)	Yes	Better than NAND, improving	Fast, non-volatile buffer enabling instant-on model loading and in-memory compute

3. Proposed Hierarchical Structure



- Store the full model library and firmware on NAND Flash for cost-effective, high-density non-volatile storage.
- Introduce an RRAM buffer tier between Flash and DRAM: on boot or model switch, weights are staged from RRAM (fast, non-volatile, low power) instead of Flash, drastically cutting model-load latency and avoiding a full flash read on every power cycle.
- Use DRAM (or on-chip SRAM for smaller models) as the active working memory for inference, holding only the currently executing model's weights and activations to minimize DRAM footprint and refresh power.
- Where feasible, exploit RRAM's compute-in-memory capability to perform matrix-vector multiplications (common in neural network inference) directly in the memory array, reducing data movement and further cutting power.

4. Expected Outcome

This Flash-RRAM-DRAM hierarchy gives near-instant model loading (RRAM avoids slow flash reads on every use), low standby power (RRAM needs no refresh), and fast inference (DRAM/SRAM for the hot path) — meeting both the speed and power budgets of an edge AI device.

Question 5: High-Performance Gaming Console

Problem: A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

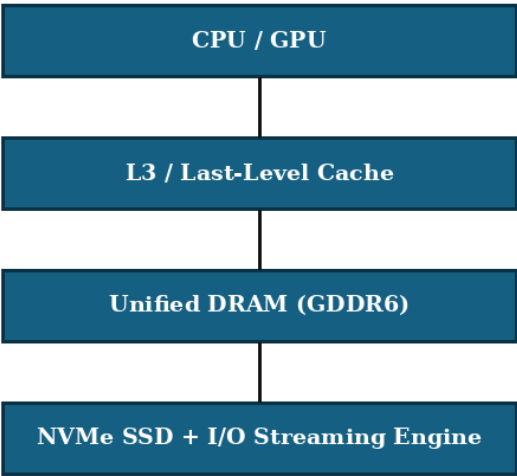
1. Understanding the Problem

Frame drops during scene loading typically occur when large textures, meshes, and level data must be streamed from storage into DRAM and then into the GPU/CPU caches faster than the current memory subsystem can sustain. This is fundamentally a throughput and bandwidth problem, compounded by cache-to-DRAM bandwidth limits when large asset sets thrash the L3/last-level cache.

2. L3 Cache vs DRAM Throughput

Aspect	L3 / Last-Level Cache	DRAM (GDDR6/DDR5)
Bandwidth	Very high (TB/s on-die) but small capacity (a few MB–tens of MB)	High (100s of GB/s) but shared across CPU, GPU, and I/O
Behavior under scene load	Rapidly thrashed by streaming new asset data, offering little reuse benefit for one-time loads	Becomes the bottleneck as large sequential texture/geometry streams saturate available bandwidth
Effect on frame rate	Cache misses stall compute pipelines momentarily	Bandwidth saturation causes sustained stalls, visible as frame drops/hitches

3. Proposed Enhancements



- Adopt a unified, high-bandwidth memory pool (as in modern consoles' GDDR6 unified memory) so the GPU is not bandwidth-starved by the CPU during simultaneous streaming and rendering.
- Use a dedicated hardware decompression/streaming engine (e.g., high-speed NVMe SSD with a custom I/O complex) so assets move directly from storage to DRAM/VRAM with minimal CPU and cache involvement, avoiding cache pollution from one-time streaming data.
- Mark streaming asset loads as 'non-temporal' / streaming stores so they bypass or minimally disturb the L3 cache, preserving cache capacity for actively reused rendering data structures.
- Increase L3 associativity/size modestly and use way-partitioning so background asset streaming cannot evict latency-critical render-pipeline data used by the GPU/CPU each frame.

- Prefetch upcoming level/scene data ahead of time (predictive streaming based on player position) so the DRAM bandwidth spike is spread over time rather than concentrated at the load boundary.

4. Expected Outcome

Separating one-time streaming traffic from cache-resident render data, combined with a higher-bandwidth unified memory pool and predictive prefetching, smooths out the DRAM bandwidth spikes that cause frame drops, resulting in seamless scene loading without stutter.